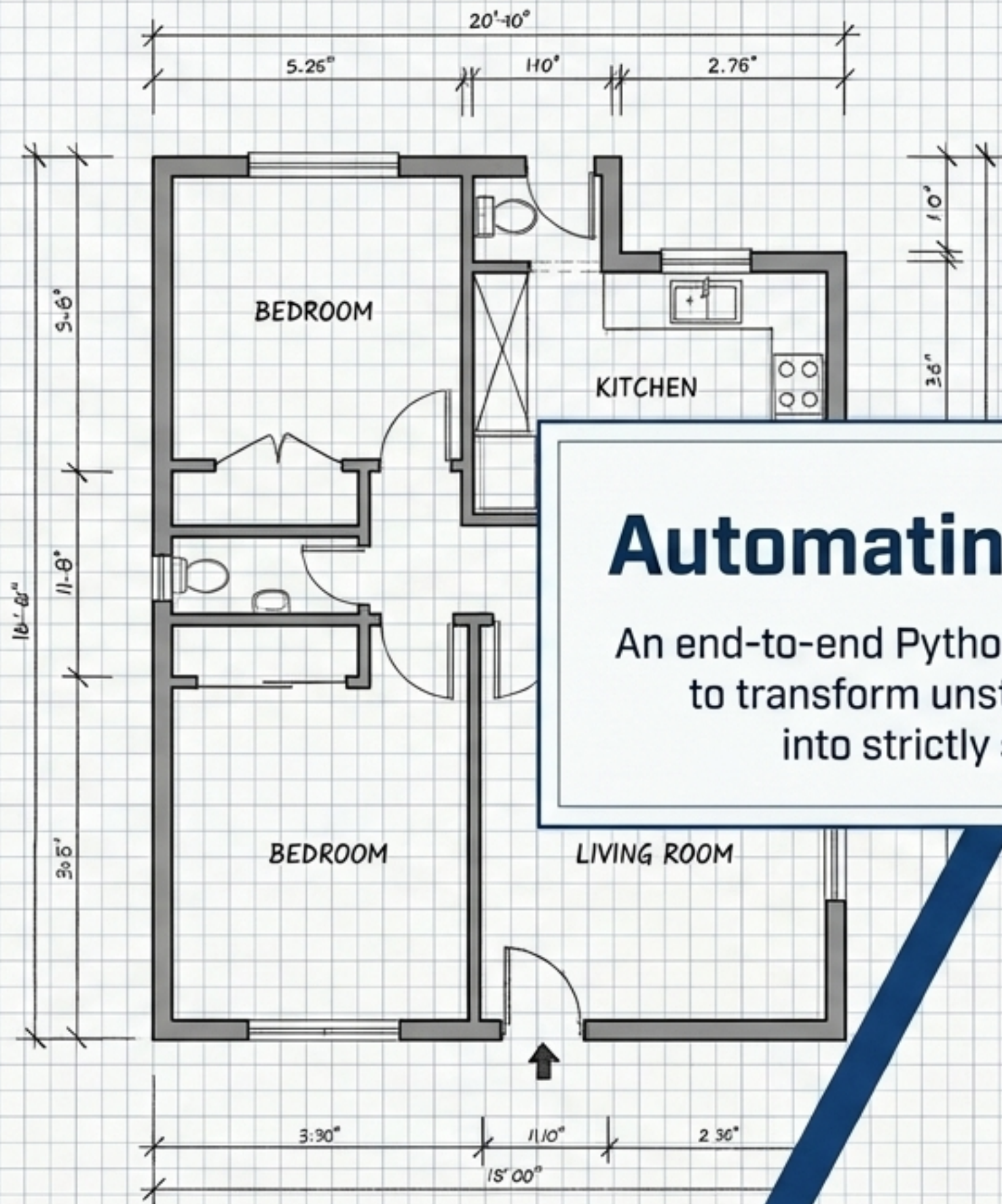




Automating Spatial Reality

An end-to-end Python and WebGL pipeline engineered to transform unstandardized 2D raster layouts into strictly structured 3D geometry.

```
{  
  "id": "window_002",  
  "type": "opening",  
  "area": 12.5, ...  
}
```

```
{  
  "id": "wall_001",  
  "type": "structural",  
  "geometry": ...  
}
```

```
{  
  "id": "room_001",  
  "type": "bedroom",  
  "area": 12.5, ...  
}
```

```
{  
  "id": "window_002",  
  "type": "opening",  
  ...  
}
```

```
{  
  "id": "room_001",  
  "type": "opening",  
  "area": 12.5,  
  "area": ...  
}
```


Traditional CAD-to-3D workflows block rapid spatial decision-making

The Status Quo

Input Requirement

Vector CAD files or specialized specialist formats



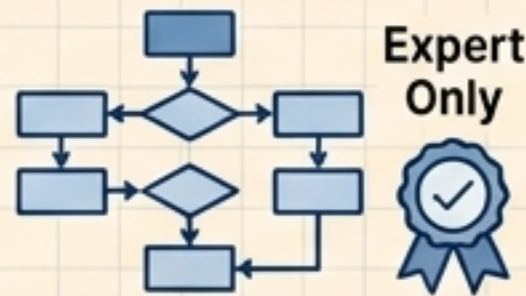
Processing Speed

Days of manual 3D modeling and correction



Operator Skill

Requires trained 3D specialists and software experts



Rendering

Relies on heavy desktop engineering software



High dependency

LOADING

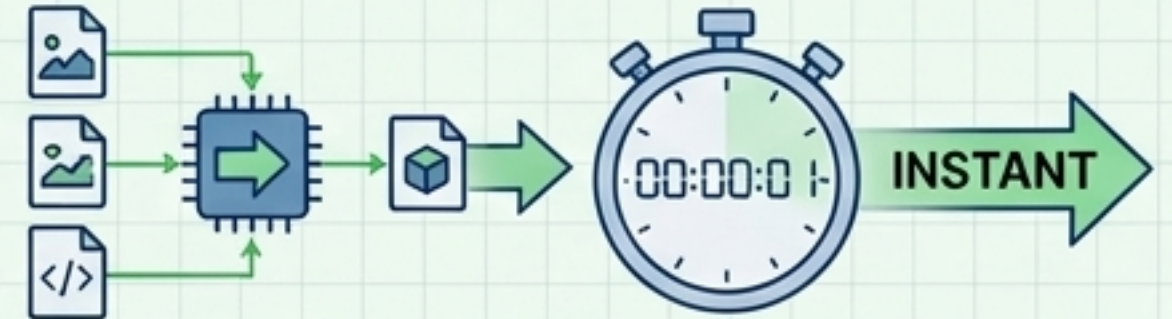


Our Pipeline

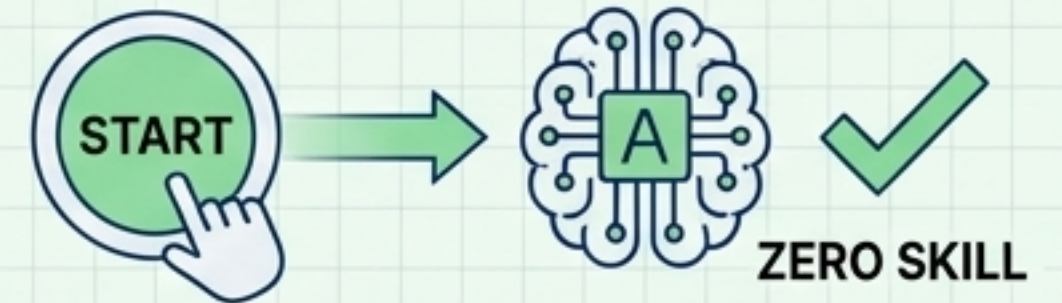
Raw PNG, JPG, and PDF rasters



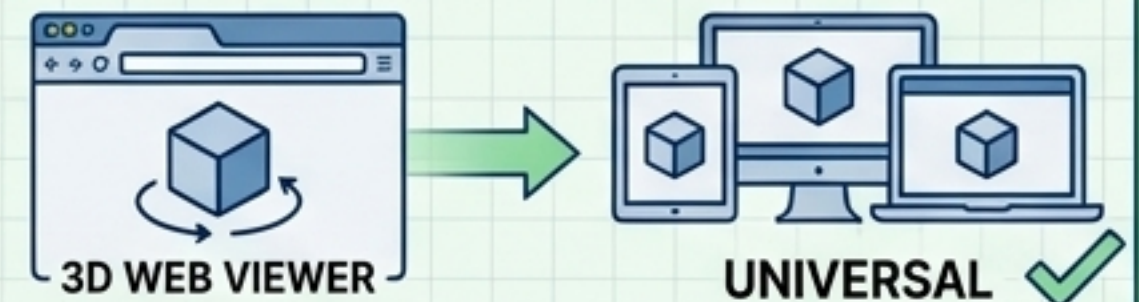
Instant, automated, and code-driven



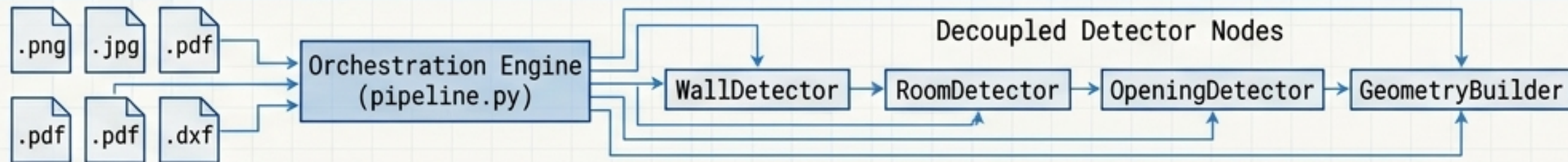
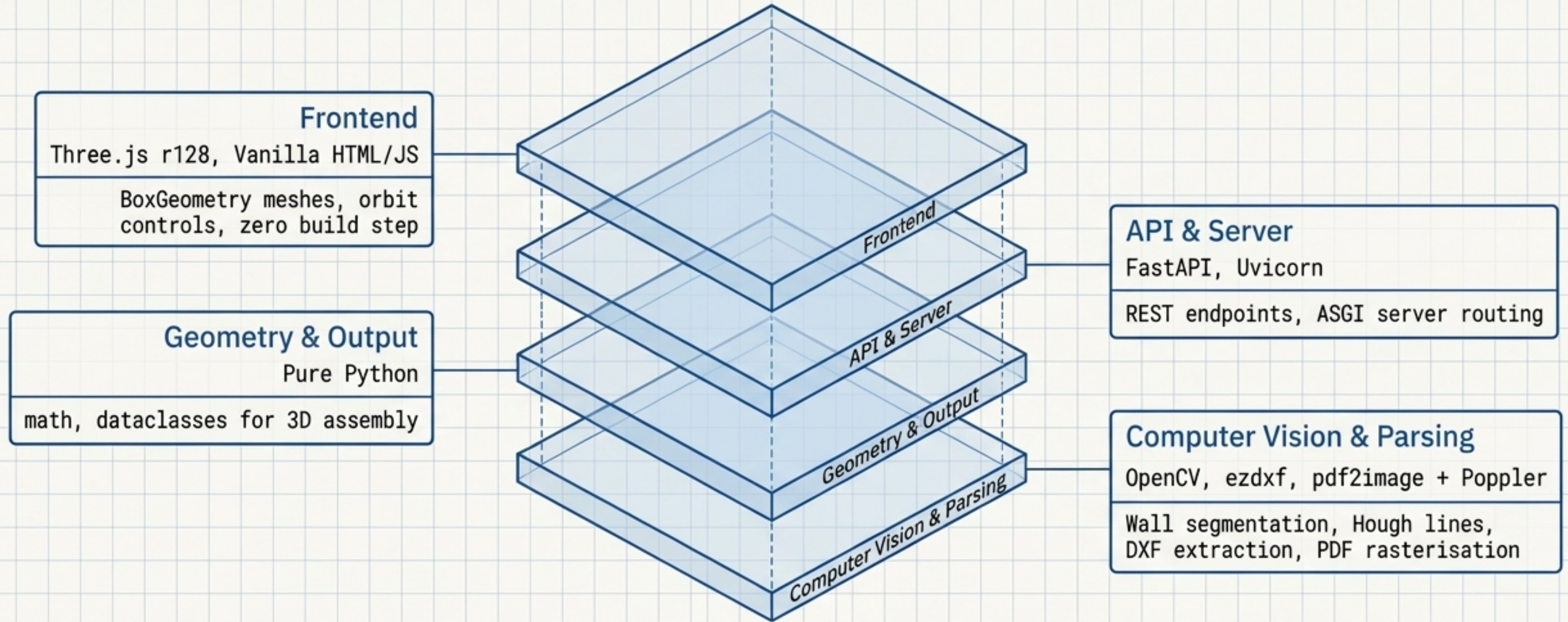
Requires zero technical skill to execute



Universally viewable, zero-dependency browser models

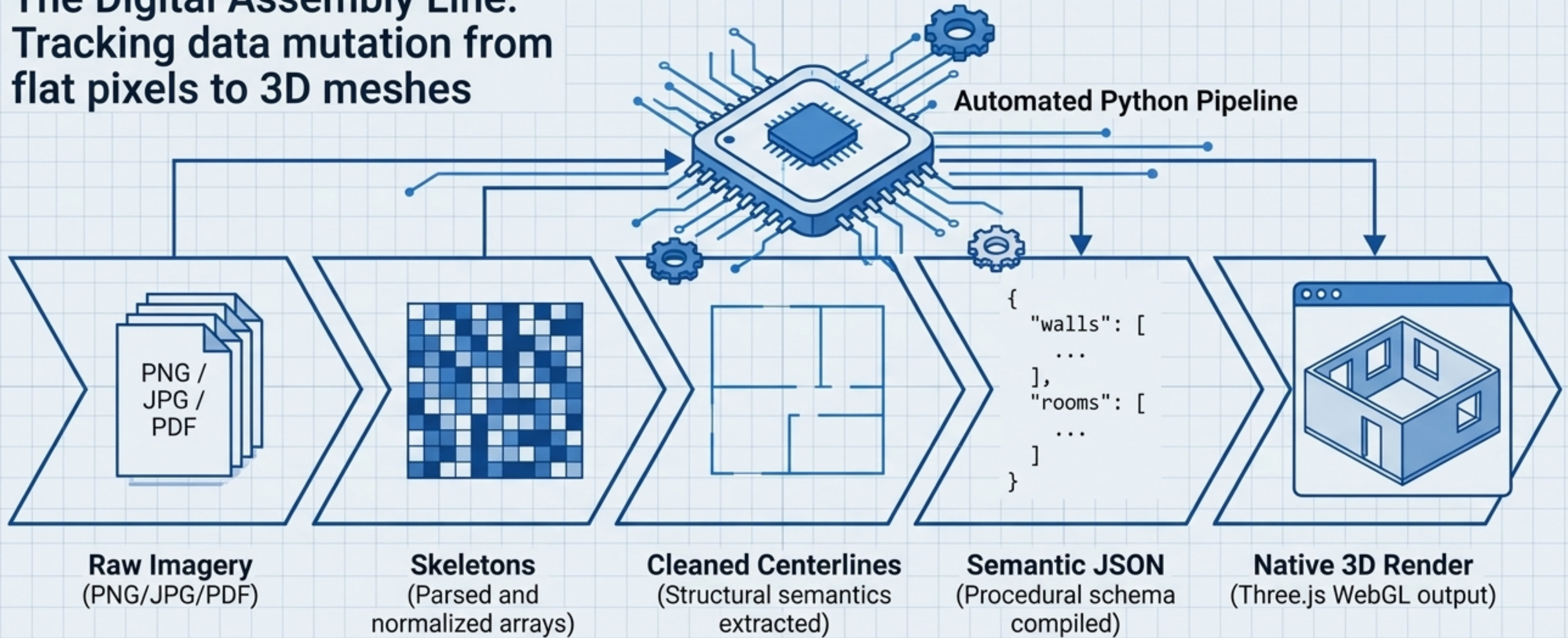


A decoupled, local-first technology stack



The orchestration engine (pipeline.py) acts as a centralized router, proving the decoupled nature of the semantic detection engines.

The Digital Assembly Line: Tracking data mutation from flat pixels to 3D meshes

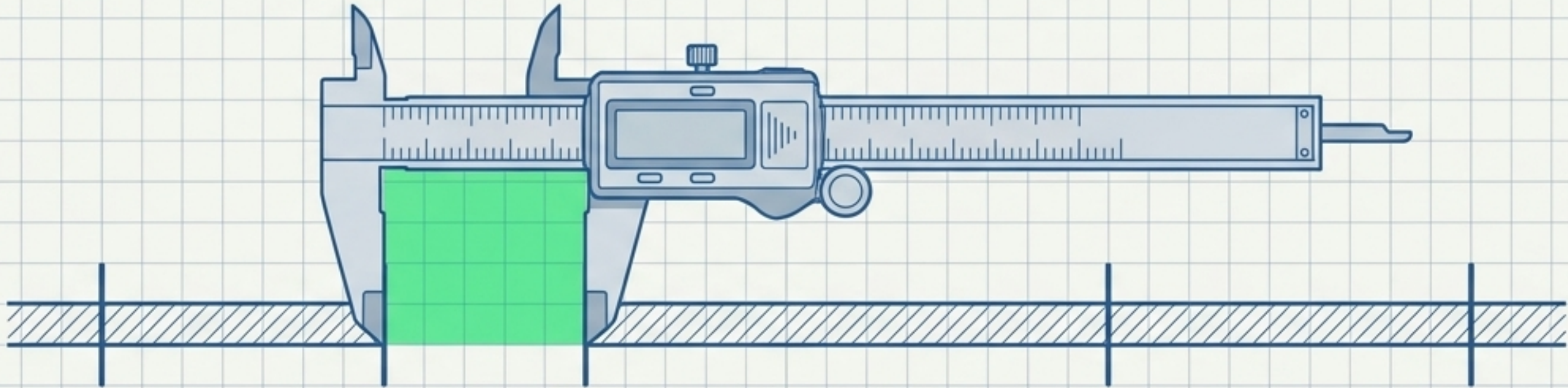


Engineering Win: The modular, staged design enables isolated debugging and controlled algorithmic tuning without rebuilding the entire execution path.

Phase 1: Smart Ingestion & Dynamic Scale Inference

Process Steps: Receive file -> Normalize to raster -> Extract Scale.

$$\frac{\text{gap_pixels}}{\text{known_meter_constant}} = \text{PPM (Pixels Per Meter)}$$

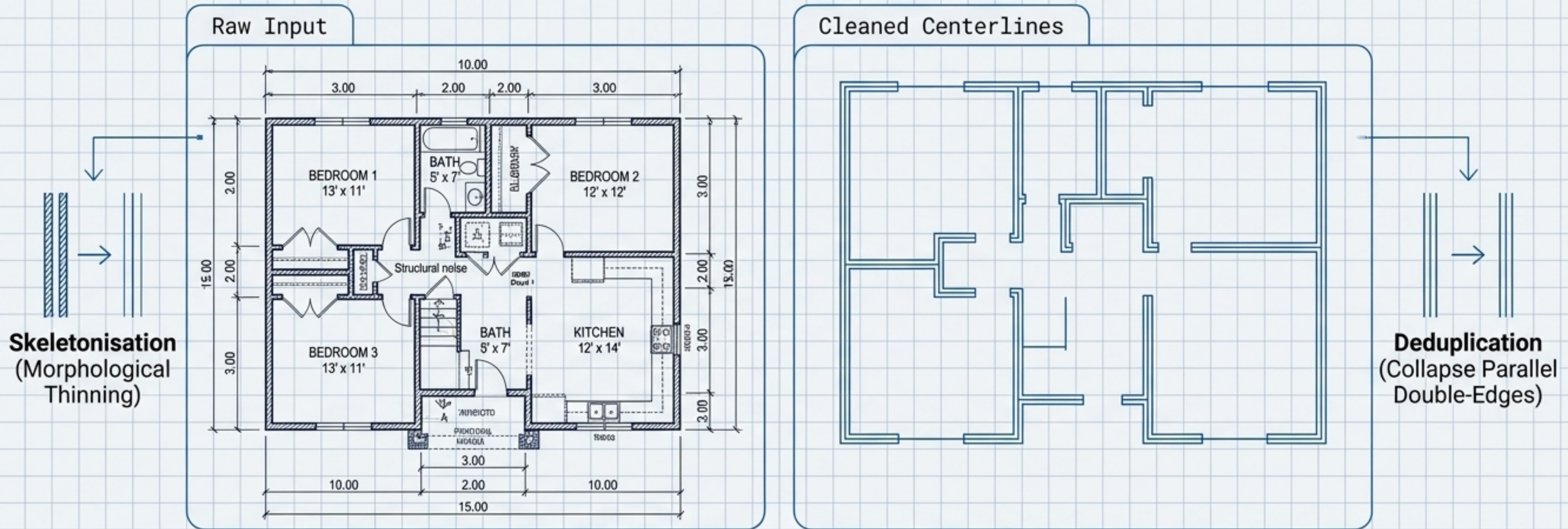


What We Did Efficiently: Dynamic PPM Detection

Instead of demanding manual scale inputs, the system actively scans for border tick marks to calculate a Pixels-Per-Meter multiplier dynamically. A robust fallback loop defaults to safe assumptions if markers are absent, guaranteeing the pipeline never crashes on missing data.

Phase 2: Structural Wall Detection & Active Noise Suppression

Process Steps: Gray-range filtering -> Hough/merge stages -> Deduplication.

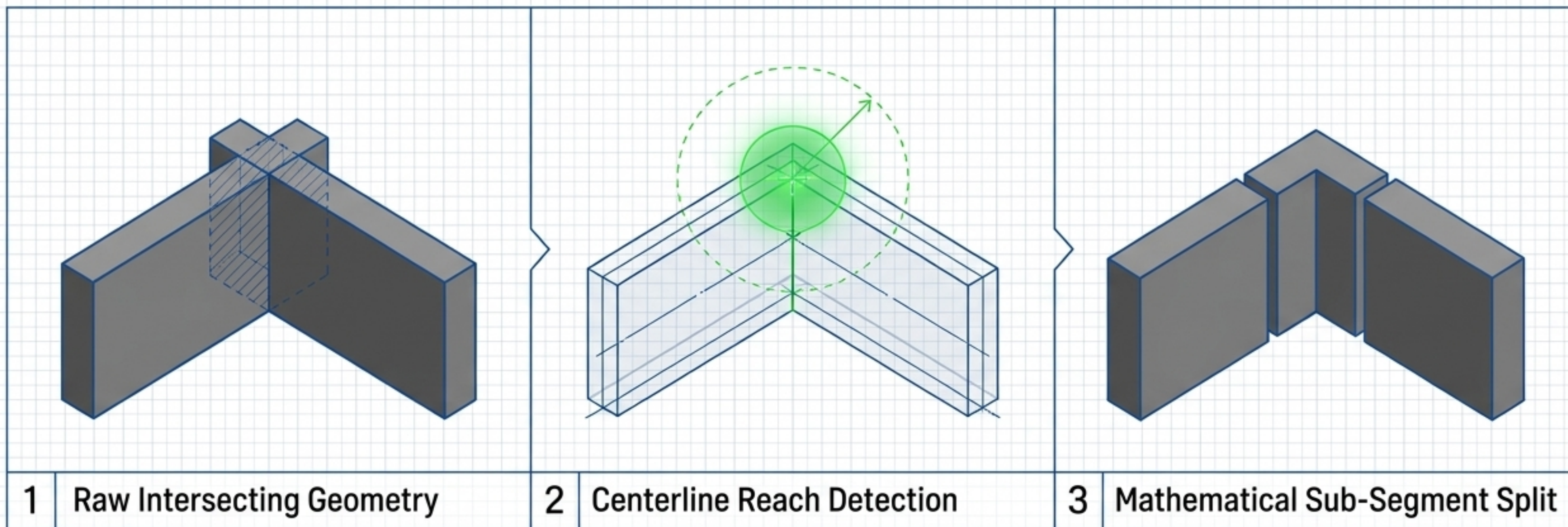


What We Did Efficiently: Adaptive Crop & Green Suppression

- **The Problem:** Text and annotations pollute Hough line detection.
- **Our Solution:** We engineered an adaptive border crop and a suppression layer (GREEN_DIFF=22) to actively delete non-structural pixels, cleanly collapsing thick double-edges into precise centerlines.

Phase 3: Reach-Based Segmentation for Junction Splitting Logic

Traditional parsers create overlapping geometry errors at corners. We built a robust geometric constraint engine.

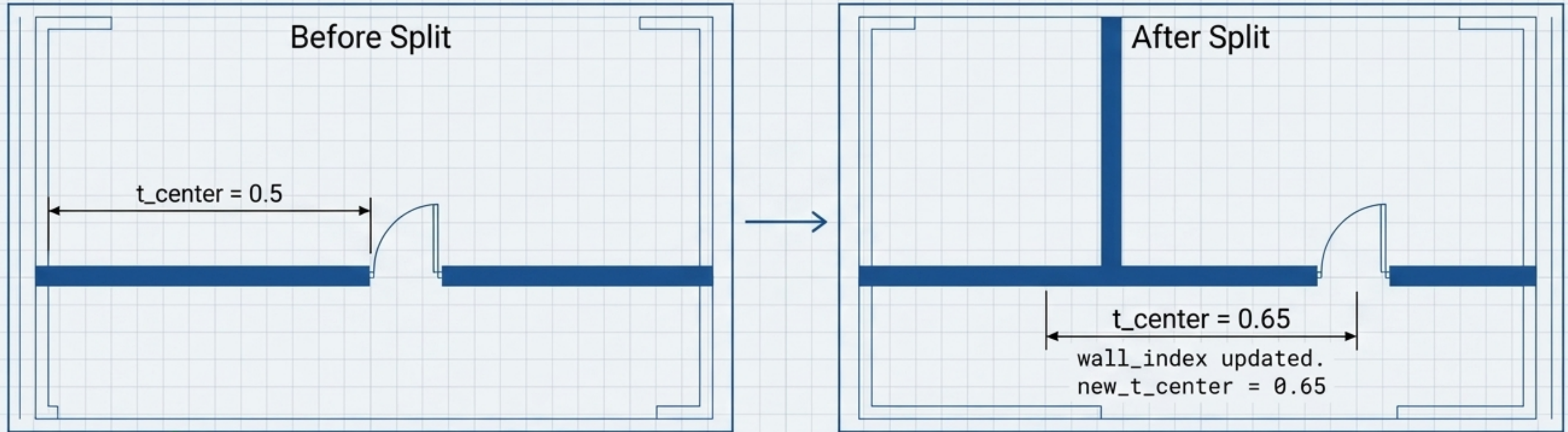


What We Did Efficiently: Reach-Based Segmentation

The algorithm identifies wall-to-wall reach interactions ($t_b * L_b \in [-reach, L_b + reach]$) and actively splits walls at valid T/+ junctions. It preserves all dimensional attributes within the newly created subsegments, readying the geometry for 3D extrusion.

Phase 4: Intelligent Opening Reprojection preserves placement fidelity

The Technical Challenge: When walls are split at junctions, attached openings (doors/windows) risk shifting or breaking entirely.



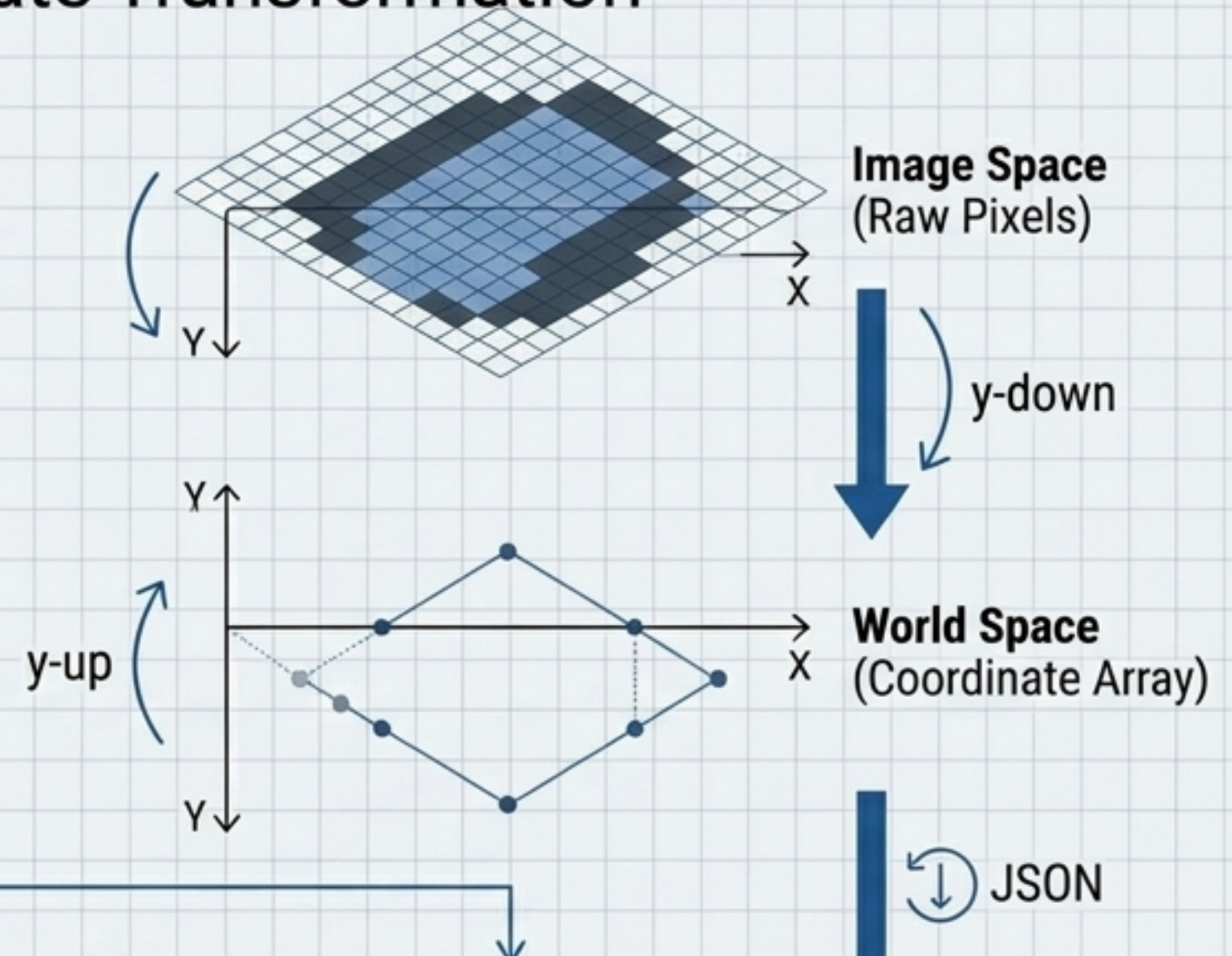
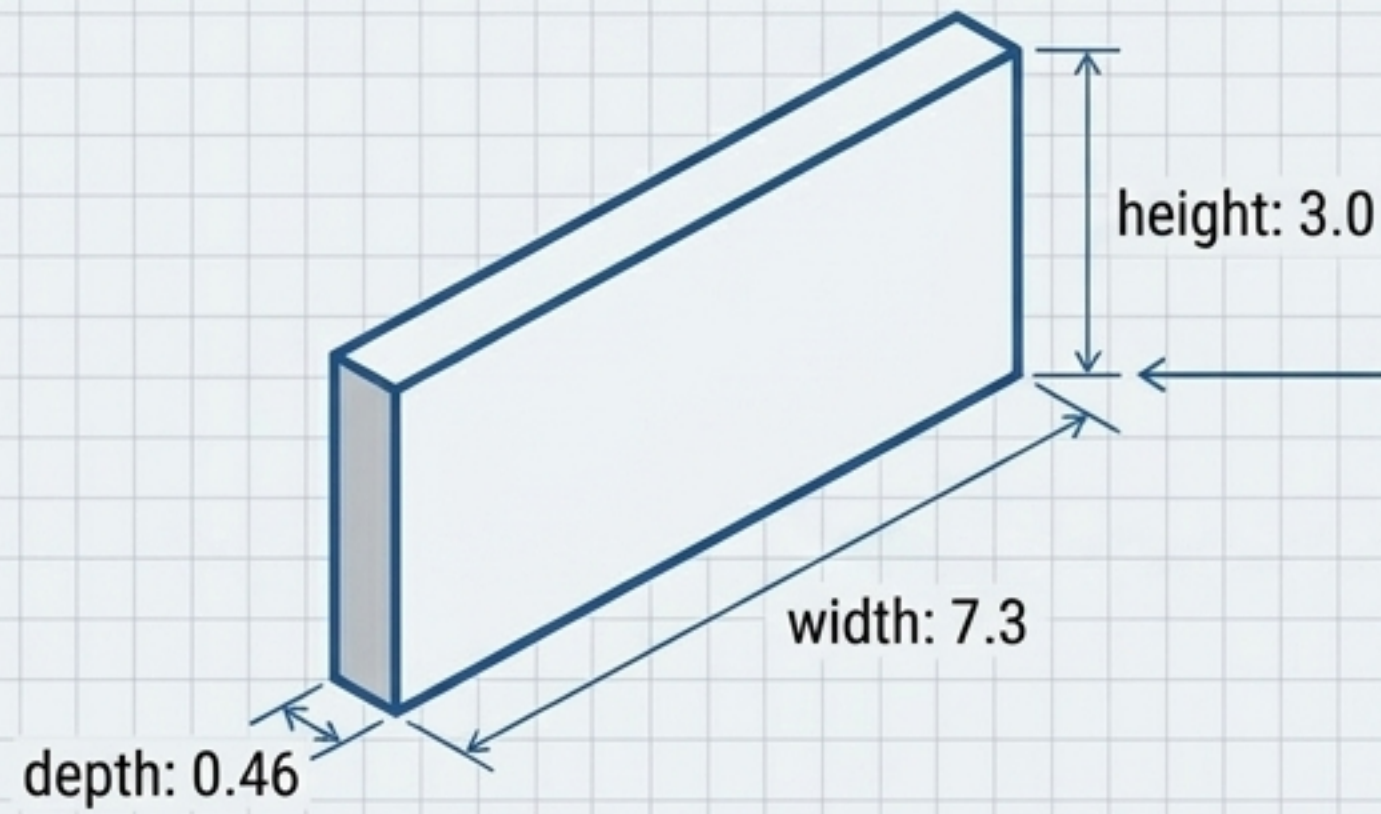
What We Did Efficiently: The t_{center} Parameterization

Openings are not mapped via absolute coordinates. They are attached to wall indices via a normalized parametric position (t_{center}).

When a wall splits, our reprojection logic dynamically recalculates the opening's position onto the new sub-wall, ensuring zero placement fidelity loss.

Phase 5: Geometry Compilation & Strict Coordinate Transformation

Process: Distinguishing room fill via threshold bands
-> Building enclosing spatial logic.



⚙️ What We Did Efficiently: Coordinate Transformation Engine

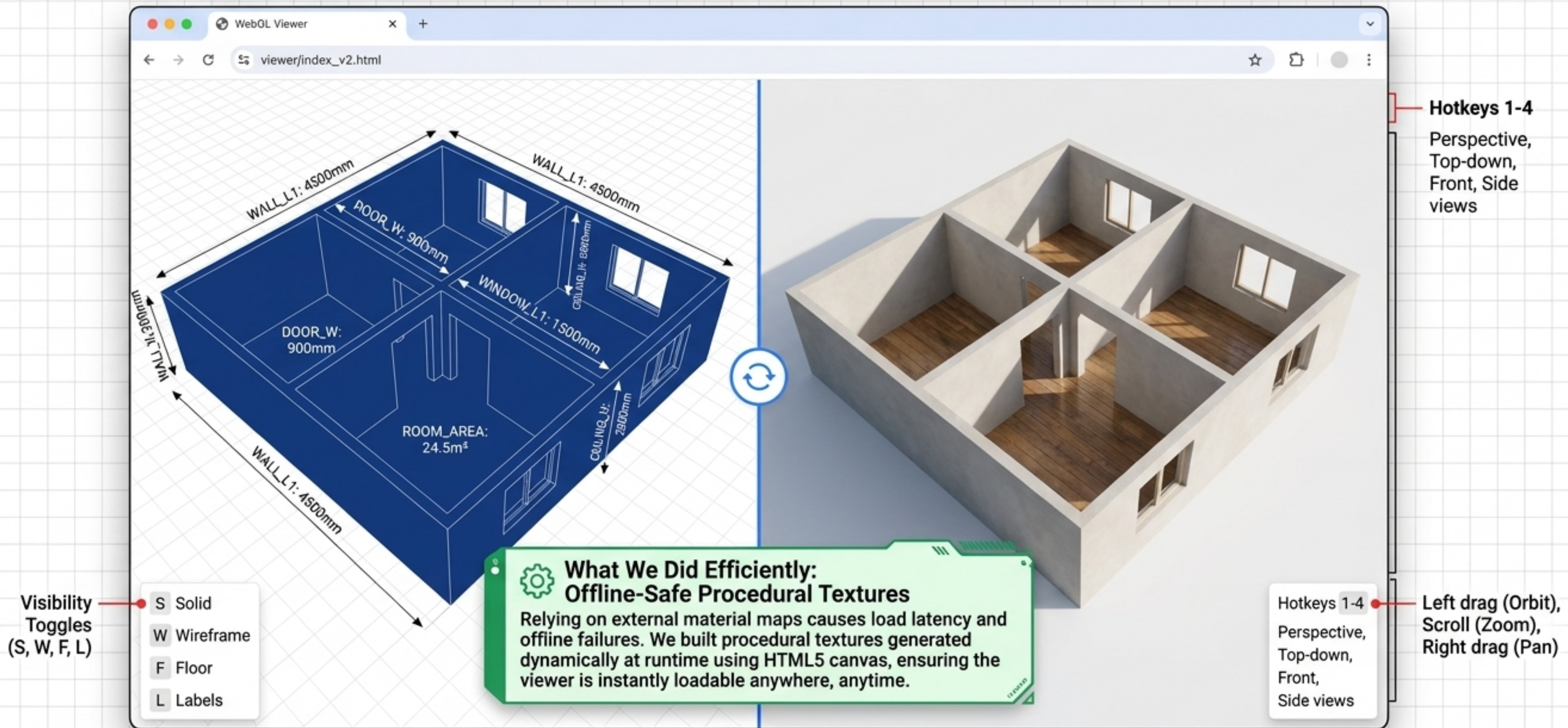
Image coordinates inherently map as y-down. Our geometry builder flips and transforms these into y-up World Coordinates, outputting a strictly standardized JSON building model schema perfectly mapped for Three.js depth constraints.

```
{
  "id": "W1",
  "type": "wall",
  "coordinates": [
    { "x": 18.5, "y": 0.0, "z": 15.2 },
    { "x": 25.8, "y": 0.0, "z": 15.2 },
    { "x": 25.8, "y": 3.0, "z": 15.2 },
    { "x": 18.5, "y": 3.0, "z": 15.2 }
  ],
  "dimensions": {
    "width": 7.3,
    "height": 3.0,
    "depth": 0.46
  }
}
```

Render-Ready Space
(JSON Schema)

Phase 6: Performant, Zero-Dependency WebGL Rendering

Consumes the generated JSON schema into a custom Three.js scene. Supports instant mode switching and element picking interactions.



A single-file Vanilla HTML setup. Zero build steps.

Defensive Engineering: Diagnosing and mitigating real-world edge cases

The system wasn't just built to process a perfect plan; it was engineered to handle real-world chaos gracefully.

Defensive Engineering & Error Handling		
System Symptom	Root Cause	Algorithmic Mitigation
Hough line detection crashed on raw inputs.	Heavy text annotations polluted the arrays.	Engineered active green-suppression masks (<code>GREEN_DIFF=22</code>) to actively delete non-structural pixels.
Windows shifted laterally after a wall split.	They were using a fractional length of the original unsplit wall.	Re-projected <code>t_center</code> specifically onto the new sub-wall coordinates.
Door swings rotated incorrectly in the 3D world.	The system was using Three.js Y-rotation (<code>rot_y</code>) for math.	Replaced <code>rot_y</code> with precise wall unit vectors (<code>ux, uy</code>) for accurate world-space projection.
Render broke on differently scaled layouts.	Hardcoded wall thicknesses (e.g., <code>thick > 0.25</code>).	Replaced fixed pixels with a dynamic median-relative threshold (<code>medianThick * 1.15</code>).

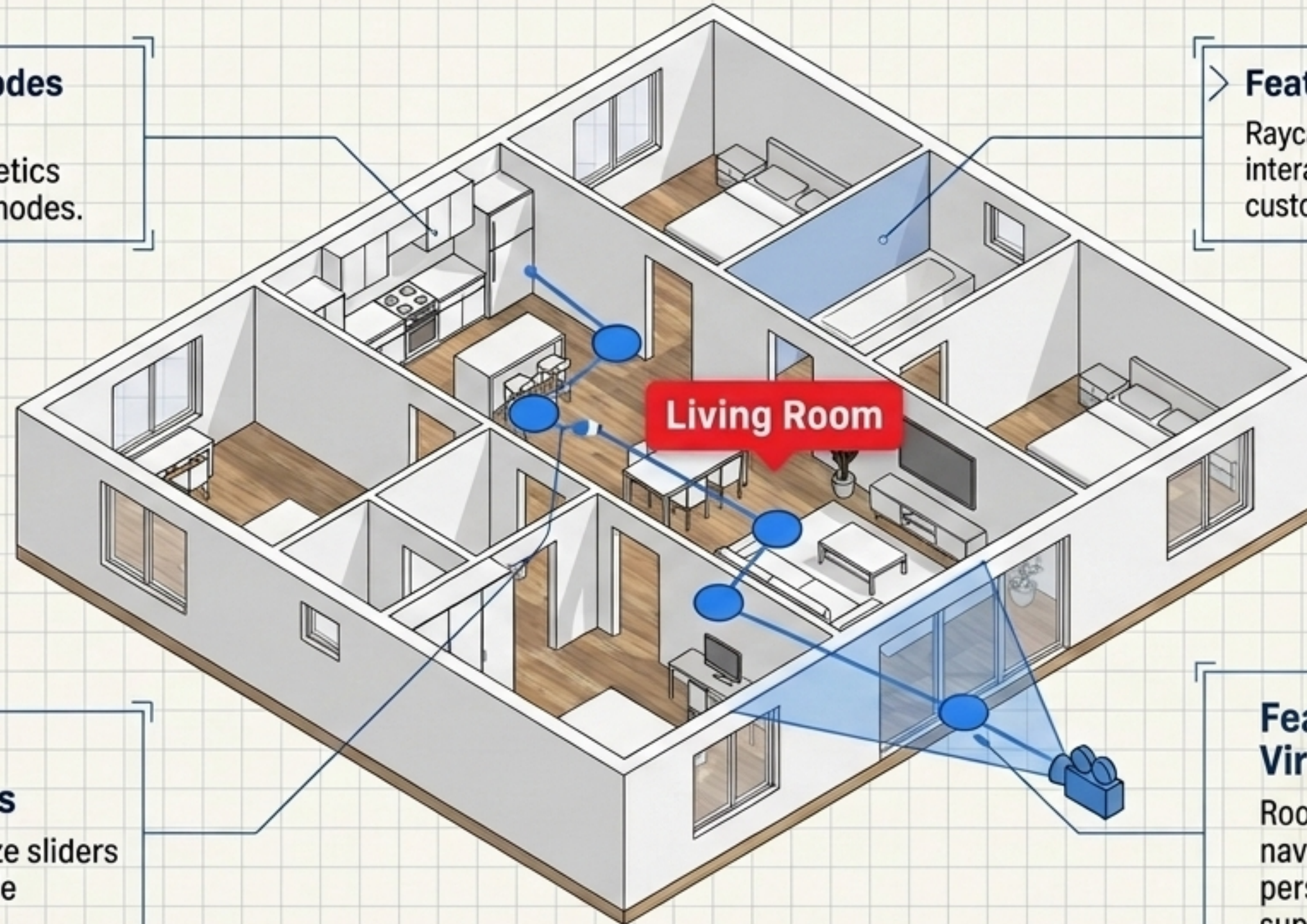
Output Capabilities: Interactive spatial reality in the browser

> Feature 1: Viewport Modes

Instant toggling between wireframe blueprint aesthetics and realistically textured modes.

> Feature 2: Element Interaction

Raycasting allows users to interactively pick individual walls for custom solid color or texture application.



Feature 3: Dynamic Adjustments

Runtime-adjustable tile size sliders control texture repeat scale dynamically.

Feature 4: Virtual & WebXR Tour Engine

Room centroids serve as calculated navigational hotspots for a first-person walkthrough. VR and webXR support giving VR tours as well.

